

UNIVERSIDAD DE LOS ANDES
Facultad de Ingeniería y Ciencias Aplicadas
Arquitectura de Computadores 2026-20

Proyecto Nº2

Juego de reflejos sobre Pochoco SoC

Grupo 10:
Juan Vicente Celedón
Ignacio Cruzat
Wille Saarenpaa

Profesor de Cátedra: Jorge Gómez M.
Ayudantes:
Fernanda Alejandra Pérez Poblete
Chen Shi Ye
Nicolás Andrés Villegas Castillo

8 de octubre de 2026

Índice

1. Introducción **2**

2. Arquitectura e implementación en FPGA **2**

2.1. Arquitectura base e implementación del juego 2

2.2. Integración del programa y los periféricos 3

2.3. Contador de ciclos 4

3. Programa assembly del juego **4**

3.1. Secuencia de una partida 4

3.2. Espera de tres segundos y selección del LED 5

3.3. Medición y conversión a décimas de segundo 6

3.4. Representación en los displays, error y promedio 6

3.5. Límites de la implementación 7

4. Assembler propio **7**

4.1. Sintaxis e instrucciones implementadas 7

4.2. Codificación, etiquetas e inmediatos 8

4.3. Validación y alcance 9

5. Verificación del proyecto **9**

5.1. Comprobación del código máquina 9

5.2. Comprobación funcional del programa 10

5.3. Síntesis e integración 10

5.4. Evidencia de funcionamiento en la FPGA 11

6. Conclusión **13**

Referencias **13**

1. Introducción

Desarrollamos un juego de reflejos en assembly para Espino Core y lo integramos en Pochoco SoC sobre la FPGA Lattice iCE40 HX1K de la Go Board. En cada ronda se encienden inicialmente los cuatro LEDs; después queda encendido un LED objetivo y el jugador debe presionar su botón asociado. El programa muestra el tiempo de respuesta en décimas de segundo y, al completar diez aciertos, presenta el promedio. Una respuesta incorrecta indica error y reinicia la ronda actual, conservando los aciertos anteriores, según lo solicitado en el enunciado [1].

Utilizamos Pochoco SoC y Espino Core como arquitectura base. Sobre ella implementamos el programa del juego, un assembler en Python, el módulo superior de integración y un contador de ciclos accesible desde el programa. En este informe explicamos la implementación, los cálculos de temporización y la traducción a código máquina; además, distinguimos las comprobaciones de los archivos de la evidencia de funcionamiento en la placa.

2. Arquitectura e implementación en FPGA

2.1. Arquitectura base e implementación del juego

Utilizamos como base Pochoco SoC, que integra Espino Core, la RAM y los periféricos mediante accesos mapeados en memoria [2]. Conservamos el núcleo, la memoria, la interfaz SPI y la asignación de pines proporcionados. La integración sigue una jerarquía de módulos: el SoC contiene el núcleo, la RAM y los periféricos, y `game_top` lo conecta con la placa.

Para ejecutar el juego implementamos el programa assembly y su assembler, extendimos el periférico con un contador de ciclos y creamos `game_top` para conectar el SoC y seleccionar la imagen de memoria. La Tabla 1 resume cómo se integran estos componentes en la solución.

Tabla 1: Componentes de la implementación del juego sobre Pochoco SoC.

Componente	Función en la implementación
`assembler/assemble.py` y `assembler/game.s`	Implementan la traducción a código máquina y la lógica del juego.
`rtl/pochoco_periph.v`	Conserva los registros de LEDs, botones y displays de la base e incorpora el contador de ciclos de 32 bits y su dirección de lectura.
`rtl/game_top.v`	Instancia Pochoco, conecta los puertos de la placa y selecciona la imagen de memoria del juego.
`Makefile`	Selecciona `game_top` como módulo superior para la síntesis.
`sw/game.hex`	Contiene las instrucciones generadas por el assembler para inicializar la memoria.

Para sintetizar y programar la FPGA utilizamos las herramientas de OSS CAD Suite [3].

2.2. Integración del programa y los periféricos

El módulo `game_top` conecta el reloj, los cuatro botones, los cuatro LEDs, los dos displays y las señales SPI con `pochoco_soc`. Mediante el parámetro `MemFile` seleccionamos `game.hex`, que inicializa la RAM al generar la configuración de la FPGA. La RAM unificada dispone de dos puertos síncronos: uno obtiene las instrucciones y otro accede a los datos. Su capacidad efectiva es de 512 palabras de 32 bits, equivalentes a $512 \cdot 32/8 = 2048$ bytes. Nuestro programa ocupa 94 palabras, es decir, $94 \cdot 4 = 376$ bytes, por lo que cabe en esa memoria.

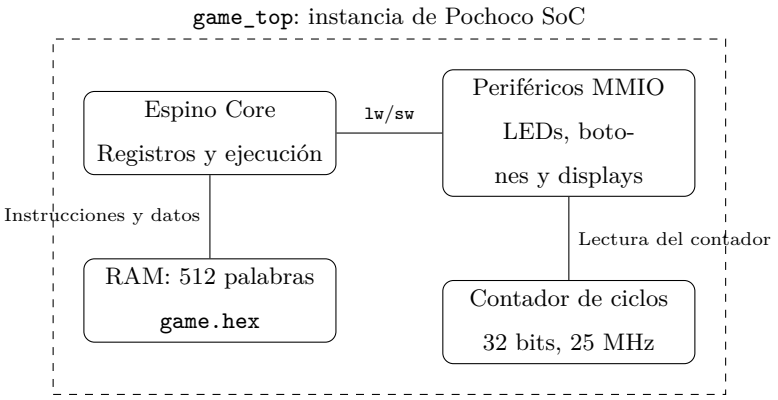


Figura 1: Organización funcional utilizada por el juego sobre la arquitectura base. El periférico incorpora el contador de ciclos utilizado para la temporización. Se omite SPI del esquema porque el juego no lo utiliza. Fuente: elaboración propia.

Los registros periféricos utilizados se muestran en la Tabla 2. Cargamos la dirección base `0x80000000`

en `x2` y accedemos a los registros mediante sus desplazamientos. Los botones se leen como una máscara de cuatro bits: la correspondencia es SW1–D1, SW2–D2, SW3–D3 y SW4–D4. Mirando la placa como aparece en las fotografías, los LEDs están debajo de los displays y se ordenan horizontalmente del 1 al 4, de izquierda a derecha. Los botones forman dos columnas: SW1 arriba y SW2 abajo a la izquierda; SW3 arriba y SW4 abajo a la derecha.

En los accesos de datos, el bit 31 de la dirección selecciona RAM cuando vale cero y MMIO cuando vale uno. Dentro de MMIO, el bit 16 distingue los periféricos de la placa, con valor cero, de SPI, con valor uno. En `pochoco_periph`, `off = addr_i[7:2]` selecciona el registro. Como una palabra de 32 bits ocupa cuatro bytes, los registros utilizados están separados por los desplazamientos 0, 4, 8 y 12; por ejemplo, el contador corresponde a `0x80000000 + 12 = 0x8000000C`.

Tabla 2: Mapa de memoria utilizado por el programa.

Dirección	Acceso del juego	Contenido
`0x80000000`	Escritura	Ocho bits para los dos dígitos del display.
`0x80000004`	Escritura	Cuatro bits de encendido de los LEDs.
`0x80000008`	Lectura	Estado de los cuatro botones.
`0x8000000C`	Lectura	Contador de ciclos de 32 bits agregado al RTL.

2.3. Contador de ciclos

Agregamos el registro `cycles_q` en `pochoco_periph.v`. El contador se reinicia a cero con el reset y aumenta en uno en cada flanco ascendente del reloj, independientemente de la instrucción que ejecute el procesador. Al leer el desplazamiento `12(x2)`, el periférico entrega su valor al núcleo. Por lo tanto, contamos ciclos de reloj, no instrucciones ni iteraciones de un bucle.

La Go Board utiliza un reloj nominal de 25 MHz [4], conectado directamente al SoC, sin un divisor adicional. El período de reloj es:

$$T_{\text{clk}} = \frac{1}{25\,000\,000} = 40 \text{ ns}. \tag{1}$$

3. Programa assembly del juego

3.1. Secuencia de una partida

Organizamos el programa en etiquetas que representan sus etapas. Al arrancar, cargamos `0x80000000` en `x2`, sumamos cuatro para apuntar a `0x80000004` y escribimos la máscara $15 =$ `0xF`, que enciende los cuatro LEDs. Luego restamos cuatro para restaurar la dirección base en `x2`. Después inicializamos `x10`, que cuenta

los aciertos, y `x12`, que acumula sus tiempos en décimas; ambos comienzan en cero. La máscara del LED objetivo se guarda en `x13`. La preparación de cada ronda vuelve a escribir la máscara de los cuatro LEDs y conserva la espera programada. El flujo implementado se resume en la Tabla 3.

Tabla 3: Etapas del programa `game.s`.

Etiqueta	Acción y transición
`round`, `wait3`	Si hay diez aciertos, pasa al promedio. En otro caso enciende los cuatro LEDs y espera el plazo de tres segundos.
`show_led`, `wait_press`	Enciende sólo el LED seleccionado, guarda el contador inicial y consulta los botones hasta una respuesta.
`div_l`, `bcd1`	Ante un acierto, convierte los ciclos a décimas, acumula el resultado, lo muestra y aumenta la cuenta de aciertos.
`wrong`	Ante una máscara incorrecta, apaga los LEDs y escribe `EE`; conserva la suma y la cuenta de aciertos.
`wait_rel`, `wait_rel_b`	Espera que se liberen todos los botones y comienza otra ronda.
`done`, `hang`	Calcula el promedio de diez aciertos, lo muestra y permanece en un bucle final.

Una lectura de botones igual a cero significa que no hay respuesta. Cualquier máscara no nula debe coincidir exactamente con el LED objetivo para contar como acierto; pulsar un botón diferente o varios a la vez produce error. La espera de liberación evita que una pulsación mantenida al terminar una ronda se cuente nuevamente. El display conserva su último valor durante la preparación de la ronda siguiente, incluyendo `EE` después de un error.

3.2. Espera de tres segundos y selección del LED

La cantidad de ciclos requerida para tres segundos es:

$$N_{3s} = 3 \cdot 25\,000\,000 = 75\,000\,000. \tag{2}$$

Después de escribir la máscara `0xF` para encender los cuatro LEDs, leemos el contador y construimos una marca límite sumándole N_{3s}. El bucle `wait3` consulta el contador hasta alcanzar esa marca. La constante se carga usando `lui` y dos `addi`, ya que no cabe en el inmediato de 12 bits de una sola instrucción:

$$\texttt{0x4786} \cdot 2^{12} + \texttt{0x7FF} + \texttt{0xC1} = 75\,000\,000. \tag{3}$$

Para seleccionar el objetivo inicializamos `x11` en uno y lo incrementamos en los bucles de espera `wait3` y `wait_press`. Luego usamos `andi x4, x11, 3` para extraer sus dos bits inferiores. Los resultados 0, 1, 2 y 3

seleccionan las máscaras 1, 2, 4 y 8, respectivamente.

Este mecanismo hace depender las selecciones posteriores del número de iteraciones de espera, incluido el tiempo que demora el jugador. Parte de una semilla fija: el primer objetivo tras un arranque idéntico es determinista. El tiempo de respuesta puede alterar el estado de `x11` y, con ello, los objetivos de rondas posteriores. El requisito del enunciado se refiere a la variación perceptible de la secuencia completa entre partidas; una primera selección repetida no basta para concluir que toda la secuencia sea fija.

3.3. Medición y conversión a décimas de segundo

En `show_led` escribimos la máscara del objetivo en los LEDs y, a continuación, leemos el contador para guardar el inicio. Cuando detectamos una respuesta correcta, volvemos a leerlo y restamos ambos valores:

$$C_i = (C_{\text{fin},i} - C_{\text{inicio},i}) \bmod 2^{32}. \tag{4}$$

Una décima de segundo corresponde a:

$$N_{0,1s} = \frac{25\,000\,000}{10} = 2\,500\,000 \text{ ciclos}. \tag{5}$$

Construimos esta constante como $\texttt{0x262} \cdot 2^{12} + \texttt{0x5A0}$. El programa divide C_i por 2 500 000 mediante restas sucesivas e incrementa un cociente por cada resta. No utiliza una instrucción de división. Para $0 \le C_i < 2^{31}$, la fracción restante se descarta y el resultado se limita a 99 décimas antes de acumularlo:

$$d_i = \text{mín}\left(99, \left\lfloor \frac{C_i}{2\,500\,000} \right\rfloor\right). \tag{6}$$

Por ejemplo, 17 250 000 ciclos representan 0,69 s, pero el programa registra 6 décimas y muestra `06`, equivalente a 0,6 s. El máximo representable es `99`, equivalente a 9,9 s; dentro del intervalo indicado, los tiempos mayores quedan limitados a ese valor. La resolución mostrada es de una décima, aunque el contador avance cada 40 ns.

3.4. Representación en los displays, error y promedio

El periférico base decodifica por separado los dos nibbles del registro del display. Para representar un valor decimal d_i de dos cifras, separamos sus decenas y unidades mediante restas de diez y formamos:

$$B_i = 16 \left\lfloor \frac{d_i}{10} \right\rfloor + (d_i \bmod 10). \tag{7}$$

Calculamos la multiplicación por 16 mediante cuatro duplicaciones con `add`. Así, 12 décimas se codifican como `0x12`, y los displays muestran los dígitos `12`, que interpretamos como 1,2 s. La unidad es una décima de segundo; por ello, los dígitos `19`, `38` y `16` de las fotografías representan 1,9, 3,8 y 1,6 s, respectivamente.

Después de cada acierto sumamos d_i en `x12` e incrementamos `x10`. Ante un error, escribimos cero en los LEDs y `0xEE` en el display. Esa rama no modifica la cuenta ni la suma; después de soltar los botones se repite la ronda pendiente.

Al llegar a diez respuestas correctas calculamos, nuevamente por restas sucesivas, el cociente entero:

$$D = \left\lfloor \frac{\sum_{i=1}^{10} d_i}{10} \right\rfloor . \tag{8}$$

Por lo tanto, promediamos los valores ya truncados y limitados de cada ronda, no los ciclos originales.

Después del décimo acierto, el programa todavía permanece en `wait_rel` mientras el botón está presionado y muestra el tiempo de ese último intento. Cuando se libera, vuelve a `round`, detecta los diez aciertos y pasa a `done`, donde muestra el promedio D. Luego permanece en `hang`. El LED objetivo de la última reacción queda encendido junto al promedio, como se observa en las fotografías finales. Para iniciar una nueva partida se vuelve a alimentar la placa. La indicación del README de que no queda ningún LED encendido no coincide con el comportamiento de esta versión.

3.5. Límites de la implementación

La activación del LED y la lectura inicial del contador se ejecutan en instrucciones consecutivas, por lo que existe una pequeña separación de ejecución que no cuantificamos. Los botones se leen por nivel, sin un filtro antirrebote agregado ni detección de flanco; un botón ya presionado al aparecer el objetivo puede aceptarse inmediatamente si coincide.

Además, `wait3` compara con `blt` una marca límite absoluta, interpretando los valores con signo. Si la marca cruza 2^{31} mientras la lectura sigue siendo positiva, el bucle puede terminar antes de los tres segundos. Con 25 MHz esto afecta a rondas que comienzan aproximadamente entre 82,9 y 85,9 s desde el reset. El bucle de división también usa `blt`: una diferencia con el bit 31 activo se interpreta como negativa y produce cero décimas en lugar de saturarse a 99. El contador completo desborda a los 171,8 s. La versión entregada no resuelve todas estas transiciones; el cálculo nominal de 75 000 000 ciclos no basta para garantizar la espera en cualquier instante de operación.

4. Assembler propio

4.1. Sintaxis e instrucciones implementadas

Desarrollamos `assemble.py` en Python para recibir `game.s` y generar `game.hex`, sin invocar un assembler RISC-V externo. Su alcance es el subconjunto que utiliza el juego, con registros `x0` a `x15`, correspondiente al banco de registros de RV32E. Los campos de registro conservan cinco bits en la codificación, aunque sólo aceptamos índices de 0 a 15. La Tabla 4 reúne sus once instrucciones.

Tabla 4: Instrucciones codificadas por nuestro assembler.

Formato	Instrucciones	Función en el juego
U	`lui`	Cargar partes altas de direcciones y constantes.
I	`addi`, `andi`, `lw`	Actualizar registros, seleccionar el objetivo y leer periféricos.
R	`add`, `sub`	Sumar tiempos y calcular diferencias.
S	`sw`	Escribir LEDs y displays.
B	`beq`, `bne`, `blt`	Comparar valores y controlar los bucles.
J	`jal`	Saltar entre etapas y volver a los bucles.

La sintaxis acepta registros escritos como `xN`, el alias `zero` para `x0`, inmediatos decimales o hexadecimales y etiquetas terminadas en dos puntos. Los accesos a memoria usan `desplazamiento(rs1)`, sin espacios dentro del operando. Se permiten comentarios con `#` o `//`. Las líneas que comienzan con un punto se omiten; en nuestro archivo esto permite conservar `.section .text` y `.global _start`, sin implementar sus funciones como directivas generales.

```
loop:
    lw   x7, 12(x2)      # leer contador de ciclos
    addi x11, x11, 1
    blt  x7, x8, loop
```

4.2. Codificación, etiquetas e inmediatos

Realizamos la traducción en dos pasadas. En la primera eliminamos comentarios, registramos las etiquetas y guardamos las instrucciones. La dirección inicial es cero y aumenta cuatro bytes por instrucción, porque cada palabra tiene 32 bits. En la segunda combinamos los campos opcode, registros, `funct3`, `funct7` e inmediato según el formato. Para un salto a una etiqueta calculamos:

$$\Delta = \text{dirección del destino} - \text{dirección de la instrucción}. \tag{9}$$

Los desplazamientos se expresan en bytes. Este procedimiento resuelve tanto etiquetas anteriores como posteriores al salto. Los formatos I y S utilizan inmediatos de 12 bits; `lui` recibe 20 bits y los coloca en la parte alta del registro destino. Para inmediatos negativos usamos su representación en complemento a dos y conservamos los bits del campo correspondiente.

Como ejemplo real del juego, `beq x10, x5, done` está en la dirección 36 y `done` en la 292. Su desplazamiento es $292 - 36 = 256$ bytes. El inmediato del formato B se distribuye entre los campos de la Tabla 5.

Tabla 5: Campos de `beq x10, x5, done`.

imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode
0	001000	00101	01010	000	0000	0	1100011

Al unir estos bits obtenemos `10550063`, que coincide con la décima palabra de `game.hex`. La salida contiene ocho cifras hexadecimales por línea, una palabra de 32 bits, en el formato que utiliza `$readmemh` para inicializar la memoria.

4.3. Validación y alcance

El assembler rechaza instrucciones no implementadas, registros fuera de `x0`–`x15`, operandos de memoria que no reconoce y desplazamientos impares en branches y `jal`. Los errores de codificación indican la línea de origen. La comprobación de desplazamiento par no verifica por sí sola la alineación de cuatro bytes; las instrucciones y etiquetas de `game.s` sí quedan en direcciones múltiplos de cuatro. No comprueba todos los casos posibles: no valida el rango de los inmediatos, la duplicación de etiquetas ni la cantidad exacta de operandos. Una máscara ajusta los inmediatos a la anchura del campo, por lo que un valor fuera de rango puede truncarse sin producir error. En `game.s` utilizamos etiquetas únicas y constantes que caben en sus campos.

Para regenerar la imagen desde el directorio raíz del proyecto ejecutamos:

```
python assembler/assemble.py assembler/game.s assembler/game.hex
```

Luego copiamos esa imagen a `pochoco_soc-main/sw/game.hex` y volvemos a sintetizar el diseño. El Makefile de los ejemplos de la base sigue usando las herramientas GNU originales; la generación del programa del juego utiliza nuestro script de Python. El enunciado solicita repetir este flujo al modificar una constante durante la demostración, regenerando la imagen con nuestro assembler y volviendo a programar la placa.

5. Verificación del proyecto

5.1. Comprobación del código máquina

Para la revisión de los archivos regeneramos el programa completo. El resultado contiene 94 palabras, equivalentes a 376 bytes, y coincide byte por byte con ambas copias de `game.hex` incluidas en el proyecto. También verificamos por decodificación independiente que los 21 saltos condicionales e incondicionales llegan a sus etiquetas previstas. Los programas pequeños `prueba.s` y `ledtest.s` reproducen sus imágenes incluidas.

Contrastamos las codificaciones de las once instrucciones implementadas con palabras esperadas, incluyendo inmediatos negativos y saltos hacia atrás. La Tabla 6 muestra cuatro casos representativos.

Tabla 6: Casos de codificación comprobados.

Instrucción	Palabra esperada y obtenida	Resultado
`lui x2, 0x80000`	`80000137`	Coincide
`addi x1, x0, -1`	`fff00093`	Coincide
`sw x13, 4(x2)`	`00d12223`	Coincide
`jal x0, -12`	`ff5ff06f`	Coincide

Los casos de rechazo ensayados incluyen una instrucción `mul`, el registro `x16`, un operando de memoria inválido y saltos con desplazamiento impar. Estas comprobaciones respaldan el subconjunto utilizado; no eliminan los límites de validación descritos en la sección anterior.

5.2. Comprobación funcional del programa

Para revisar la lógica ejecutamos un modelo funcional de las instrucciones de la fuente, con lecturas de periféricos programadas y esperas aceleradas. El modelo permite comprobar las sumas, los saltos y las escrituras; no es una simulación RTL, no reproduce el tiempo de cada instrucción ni reemplaza la demostración física. La Tabla 7 resume los resultados obtenidos en esta revisión.

Tabla 7: Resultados de la comprobación funcional con entradas controladas.

Entrada controlada	Resultado observado en el modelo
Diez aciertos con $d_i = 6, 4, 7, 12, 3, 9, 2, 5, 8, 10$	Diez aciertos, suma 66 y promedio `06`.
Un acierto de seis décimas, un error y nueve aciertos más	El error escribe `EE` y conserva un acierto y la suma 6; luego se completan los diez aciertos.
Diez respuestas de 0,69 s	Cada respuesta se trunca a seis décimas y el promedio es `06`.
Diez respuestas de 30 s	Cada respuesta se limita a 99 décimas y el promedio es `99`.
Marca límite que cruza 2^{31}	Se confirma que `blt` puede terminar anticipadamente la espera de tres segundos.

5.3. Síntesis e integración

El flujo utilizado parte del RTL y la imagen de memoria: Yosys sintetiza el diseño, nextpnr realiza la colocación y el enrutamiento, icepack genera el archivo de configuración e iceprog lo carga en la placa [3]. El

archivo `yosys_log.txt` incluido termina sin problemas en la comprobación CHECK y registra 1000 celdas `SB_LUT4` y 12 bloques `SB_RAM40_4K`. Corresponde al registro de síntesis previo, conservado con el mismo RTL; no acredita una nueva síntesis con la imagen final de 94 palabras. Tampoco contiene por sí solo los resultados de enrutamiento o temporización ni demuestra la partida en la FPGA.

En `game_top.v`, la ruta de `MemFile` está fijada al computador utilizado durante el desarrollo. Para reproducir el proyecto en otra ubicación debe ajustarse al archivo del juego; por ejemplo, `sw/game.hex` al ejecutar desde `pochoco_soc-main`. Además, una imagen regenerada requiere volver a sintetizar aunque el RTL no haya cambiado, porque el Makefile base no declara el HEX como dependencia de la síntesis.

5.4. Evidencia de funcionamiento en la FPGA

Incorporamos seis fotografías de la Go Board correspondientes a una ronda activa, la preparación, la aparición del objetivo, el error y la transición del último acierto al promedio. La secuencia descrita se interpreta junto con las escrituras y los bucles del programa. Los LEDs son pequeños, por lo que identificamos su posición siguiendo el orden del 1 al 4 indicado anteriormente.

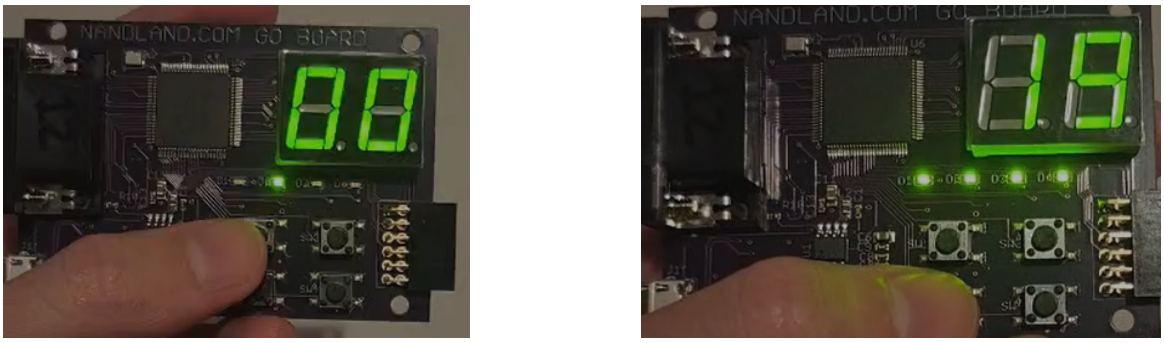


Figura 2: Izquierda: captura `Inicio`, con el LED 2 seleccionado y SW2, abajo a la izquierda, como botón asociado. Derecha: captura `Espera`, con los cuatro LEDs encendidos y los dígitos `19`, correspondientes a la reacción anterior de 1,9 s. Fuente: fotografías propias.

La imagen denominada `Inicio` muestra una ronda que ya tiene un objetivo seleccionado. En `Espera` se observa la preparación de otra ronda, antes de que se apaguen tres LEDs. El resultado anterior permanece en el display durante esa espera porque `round` sólo escribe en el registro de LEDs. La espera nominal se justifica mediante el contador y los 75 000 000 ciclos, con los límites de comparación descritos en la sección de implementación.

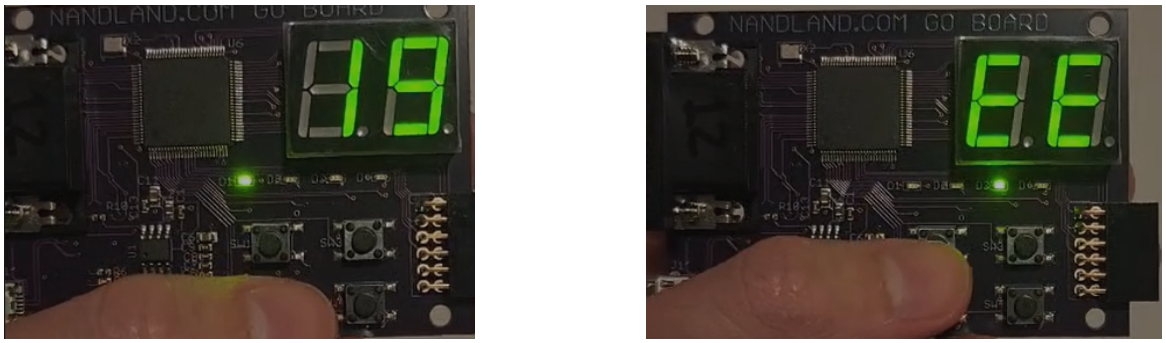


Figura 3: Izquierda: captura `Reaccionar`, con el LED 1 encendido como objetivo y el resultado anterior `19` todavía visible. Derecha: captura `Error`, con la indicación `EE`; también se aprecia el LED 3 encendido. Fuente: fotografías propias.

En `Reaccionar`, el jugador debe presionar SW1, ubicado arriba a la izquierda, tan pronto como aparezca el LED 1. Los dígitos `19` siguen correspondiendo a la reacción anterior de 1,9 s: el tiempo del nuevo intento se escribe después de detectar el acierto. Ante una respuesta incorrecta, el programa escribe `EE`, espera que se suelten los botones y repite la ronda sin modificar la cuenta ni la suma. La fotografía `Error` registra esa indicación junto a un LED encendido, situación compatible con la conservación de `EE` durante una ronda posterior, hasta que se escribe otro resultado.

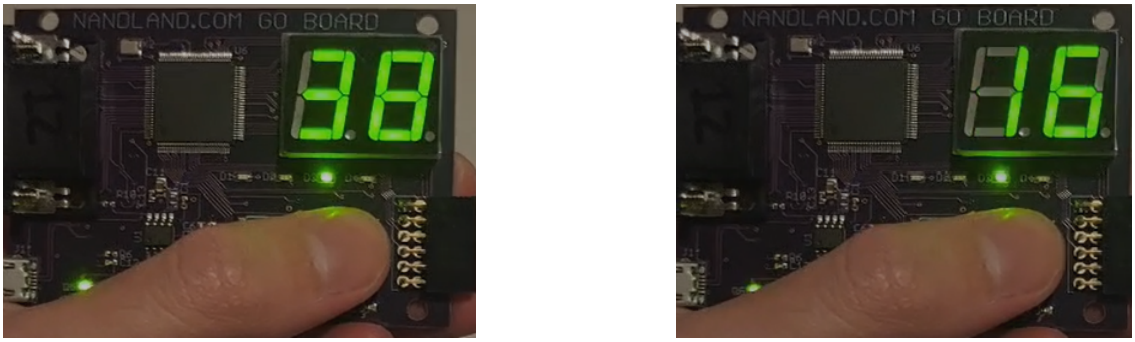


Figura 4: Izquierda: captura `Pulsacion_Final`, con el tiempo del décimo acierto, 3,8 s, mientras se mantiene presionado el botón. Derecha: captura `Promedio_Final`, con 1,6 s después de liberarlo. El LED 3, objetivo de la última reacción, permanece encendido. La transición corresponde a la secuencia registrada y descrita por el grupo. Fuente: fotografías propias.

Las fotografías finales permiten leer `38` y `16`, correspondientes al último tiempo y al promedio. Mientras el botón del décimo acierto se mantiene presionado, `wait_rel` conserva el tiempo de ese intento. Al soltarlo, se alcanza `done` y se sustituye por el promedio. El registro de LEDs no se modifica en ese recorrido, por lo que queda encendido el objetivo de la última reacción, el LED 3 en esta partida.

Los diez tiempos registrados en esta partida se presentan en la Tabla 8. El noveno acierto volvió a registrar 1,5 s y el décimo registró 3,8 s.

Tabla 8: Tiempos de respuesta de los diez aciertos de la partida registrada.

Acierto	1	2	3	4	5	6	7	8	9	10
Tiempo (s)	2,0	1,9	1,1	0,8	1,5	1,3	1,2	1,5	1,5	3,8
Décimas	20	19	11	8	15	13	12	15	15	38

Los diez valores suman 166 décimas, equivalentes a 16,6 s. Su media aritmética es $16{,}6/10 = 1{,}66$ s. El programa utiliza división entera en décimas y calcula:

$$D = \left\lfloor \frac{20 + 19 + 11 + 8 + 15 + 13 + 12 + 15 + 15 + 38}{10} \right\rfloor = \left\lfloor \frac{166}{10} \right\rfloor = 16. \tag{10}$$

Por lo tanto, muestra 16 décimas, es decir, 1,6 s, que coincide con `Promedio_Final`. El cociente entero descarta la fracción restante. Todas las lecturas de esta partida están por debajo de 99 décimas, por lo que la saturación no modifica este promedio.

Las imágenes documentan los estados visibles y la lectura final. La conservación de los aciertos ante un error se sustenta en que la rama `wrong` no modifica `x10` ni `x12`, y en la comprobación funcional de la Tabla 7. La duración nominal de la preparación se obtiene del contador y de la conversión de ciclos explicada anteriormente.

6. Conclusión

Utilizamos Pochoco SoC y Espino Core como base e implementamos el programa del juego, el assembler y su integración con un contador de ciclos accesible por memoria. El assembler traduce las once instrucciones necesarias y reproduce la imagen de 94 palabras incluida en el proyecto. La revisión funcional respalda la conversión a décimas y la conservación de aciertos ante errores. Los diez tiempos registrados suman 166 décimas y producen un promedio entero de 16 décimas, equivalente a los 1,6 s mostrados en la placa. Las fotografías también muestran el último tiempo de 3,8 s y el LED de la última reacción encendido. La implementación conserva los límites de temporización y medición descritos, que deben distinguirse del cálculo correcto del promedio de esta partida.

Referencias

[1] Universidad de los Andes. *Proyecto 2: Juego en Assembly sobre Pochoco SoC*. Enunciado de Arquitectura de Computadores, semestre 2026-2.

[2] Nicolás Villegas. *Pochoco SoC*. Repositorio base proporcionado en el curso. https://github.com/nic0villegasc/pochoco_soc.

[3] YosysHQ. *OSS CAD Suite*. Herramientas para diseño y programación de FPGA. https://github.com/YosysHQ/oss-cad-suite-build.

[4] Nandland. *The Go Board*. Características de la placa y reloj de 25 MHz. https://nandland.com/the-go-board/.

[5] Ignacio Cruzat, Juan Vicente Celedón y Wille Saarenpaa. *Proyecto2-Arqui*. Código del grupo revisado a partir del ZIP final. https://github.com/icruzat1/Proyecto2-Arqui.